Clase 18 · Planificación de tiempo real, comunicación entre procesos y RAM disk

Fundamentos de Sistemas Embebidos · M.C. José Mauricio Matamoros de María y Campos · miércoles 28 oct 2026 · Tema 3 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

Minimal Laxity First, reserva de recursos, la regla de oro de utilización, Rate Monotonic Scheduling con su cota del 70%, paso de mensajes contra memoria compartida y RAM disk. Continúa la Clase 17.

> **Fuente**
> NotebookLM (audio FSE-Clase18.m4a). Lo que no cuadra con la referencia técnica se marca con ⚠️.

1. Minimal Laxity First (MLF)

- Algoritmo **dinámico**: gana la tarea con la **menor laxitud** (holgura) en ese instante.
- **Laxitud** = tiempo restante hasta el deadline − tiempo de ejecución que aún le falta.
- Ejemplo en clase: P1 con T=50 ms y C=25 ms; P2 con T=100 ms y C=40 ms.
- En t=0: laxitud P1 = 50−25 = 25; P2 = 100−40 = 60. Gana P1.
- En t=10 ms (a P1 le quedan 15 ms y 40 ms para su entrega): P1 = 40−15 = 25; P2 = 90−40 = 50. Sigue P1 hasta t=25; luego entra P2 hasta que P1 despierta a los 50 ms. Ambas cumplen.
- **Sobrecarga del scheduler:** si calcular la fórmula cuesta tiempo real (por ejemplo 1 unidad por conmutación), ese costo se acumula y puede hacer colapsar el sistema.

Error común: confundir la laxitud con el periodo; cambia en cada instante.

2. Reserva de recursos

- Cada tarea trae un encabezado con su **periodo** y su **consumo en el peor caso (WCET)**.
- Al instanciarla, el sistema corre una simulación rápida; si la suma de demandas excede la capacidad, **rechaza** la tarea.
- Analogía: la tortillería que te dice «regrese mañana» porque la máquina está ocupada.

Error común: creer que en tiempo real las tareas se crean libremente al vuelo como en un sistema de propósito general.

3. Regla de oro de planificabilidad

Examen

La suma de las utilizaciones C/T de todas las tareas periódicas debe ser **≤ 1.0 (100%)**. El profesor la llamó «la regla de oro» y resolvió ejercicios con fracciones.

- Ejemplo: T1 (T=50, C=35) = 70/100; T2 (T=50, C=15) = 30/100; T3 (T=100, C=15) = 15/100. Suma = 1.15 (115%) → **no es planificable**; aunque la primera iteración parezca acomodarse, en el siguiente ciclo faltarán 15 ms.

Error común: revisar solo la primera iteración en una gráfica sin sumar los cocientes.

4. Rate Monotonic Scheduling (RMS)

- Algoritmo **estático** para tareas periódicas: **a menor periodo, mayor prioridad**. Las prioridades no cambian y se calculan antes de ejecutar.
- **Cota de Liu y Layland:** U ≤ n(2^(1/n) − 1).

Tareas	Cota
1	1.0 (100%)
2	≈ 0.828 (~83%)
3	≈ 0.779 (~78%)
n → ∞	ln 2 ≈ 0.693 (~70%)

- ⚠️ NotebookLM dijo que con 4 tareas «cae muy cerca del 70%»; el valor exacto con 4 tareas es ≈ 0.757. La regla práctica del profesor: con 4 o más tareas periódicas (adquisición, filtrado, ley de control, actuación), dejar ~30% de procesador libre.
- Si un conjunto no es planificable con RMS, ningún otro algoritmo de prioridad fija lo planifica en un solo procesador.

Error común: querer operar un sistema de tiempo real al 99% o 100% de uso.

5. Comunicación entre procesos

	Paso de mensajes	Memoria compartida
Idea	Carta certificada: serializar, enviar y deserializar	Pizarrón común: uno escribe, el otro lee
Costo	Muy alto (JSON, YAML, XML, sockets)	Mínimo
Dónde	Cliente-servidor, sistemas de escritorio	Estándar en embebidos (variables globales, FreeRTOS)

- Con memoria compartida hay que sincronizar para evitar condiciones de carrera: semáforos, colas y mutexes.
- En escritorio los programas se aíslan por privacidad; en embebidos todas las tareas persiguen un fin común y deben cooperar con poca latencia.

Error común: usar JSON y sockets entre hilos de un microcontrolador pequeño.

6. RAM disk

- Un bloque de RAM que el sistema operativo expone como si fuera un disco.
- Problema que resuelve en la Raspberry Pi: la MicroSD es lenta y se desgasta por ciclos de escritura.
- Se configura con un punto de montaje en `/etc/fstab`; `/tmp` en Linux suele ir en `tmpfs`. Por ser volátil no necesita bitácora (journaling) y se vacía al reiniciar.
- ⚠️ NotebookLM mezcló `tmpfs` con montar `ext2` sobre RAM; son dos mecanismos distintos.

Error común: confundir el RAM disk con la **swap** (la swap pasa RAM a disco lento; el RAM disk usa RAM para simular un disco).

Contexto — no examen

- La reprobada de Estática y la anécdota de aproximar √2 como 1.414.
- Chistes sobre «cualquier aproximación es buena» en ingeniería.
- El sistema de inscripciones: si miles de alumnos presionaran F5 a la vez; sirvió para explicar prioridades estáticas y dinámicas.
- El mural de la Facultad de Medicina tapado y el costo de entrar al estadio.
- JSON como «Jason, el cuate de Freddy Krueger».

Pendiente

- No hay clase el lunes (día feriado).
- Siguiente sesión (miércoles 4 nov): administración de memoria. Luego: relojes y temporizadores (9 nov), manejadores de dispositivos (11 nov), almacenamiento de archivos (18 nov) y administración de energía (23 nov).
- Se coordinaron las fechas de la Práctica 8 y del Proyecto Final en noviembre.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Tema	Idea para recordar
Laxitud	Tiempo al deadline − trabajo pendiente; gana la menor
MLF	Dinámico; recalcula la holgura todo el tiempo
Regla de oro	Σ C/T ≤ 1.0
RMS	Estático; menor periodo = mayor prioridad
Cota de RMS	n(2^(1/n) − 1) → ln 2 ≈ 70%
Reserva de recursos	Se rechaza la tarea que no cabe
Comunicación	Mensajes (caro) vs memoria compartida (estándar en embebidos)
RAM disk	RAM como disco: rápido y no desgasta la MicroSD

Hilo conductor

En tiempo real no basta con que las tareas «quepan»: hay que probar con la regla de oro y la cota de RMS que cumplirán sus plazos, y en un embebido todo se diseña para gastar poco procesador y poca memoria.